

u# Assignment 2.1

Name: Gunasekaran Pasupathy Date: 5/13/2026

For this assignment, you will refer to the textbook to solve the practice exercises. **Use Python to answer any coding problems (not R, even if indicated in your textbook).**

Use Jupyter Notebook, Google Colab, or a similar software program to complete your assignment. Submit your answers as a **PDF or HTML** file. As a best practice, always label your axes and provide titles for any graphs generated on this assignment. Round all quantitative answers to 2 decimal places.

Problem # 2.1.

For the rain simulation example in Section 2.1.1, but with probability of rain 0.30 on any given day, simulate the outcome (a) on the next day, (b) the next 10 days. (c) Simulate the proportion of days of rain for the next (i) 100 days, (ii) 10,000 days, (iii) 1,000,000 days. Use the simulation to explain the long-run relative frequency definition of probability.

(a) on the next day

Its very random. Definitely not 0.30

```
In [10]: import numpy as np

# Probability of rain on any given day
p = 0.30

# (a) Simulate outcome on the next day
next_day = np.random.binomial(1, p)

print("Next day:")
print("Rain" if next_day == 1 else "No Rain")
```

Next day:

Rain

b) the next 10 days:

Sometimes it is 4 out of 10, sometimes 3/10, once even got 2/10

```
In [9]: # (b) Simulate next 10 days
next_10_days = np.random.binomial(1, p, 10)
print("\nNext 10 days:")
print(next_10_days)
```

Next 10 days:

[0 0 0 0 0 1 0 1 1 0]

(c) Your answer goes here

```
In [16]: # (c) Simulate proportions
for n in [100, 10000, 1000000]:
    rainy_days = np.random.binomial(n, p)
    proportion = rainy_days / n
    print(f"\nDays simulated = {n}")
    print(f"Rainy days      = {rainy_days}")
    print(f"Proportion      = {proportion}")
```

```
Days simulated = 100
Rainy days      = 35
Proportion      = 0.35
```

```
Days simulated = 10000
Rainy days      = 2904
Proportion      = 0.2904
```

```
Days simulated = 1000000
Rainy days      = 300886
Proportion      = 0.300886
```

Problem # 2.2.

Data analysts often implement statistical inference methods by setting the probability of a correct inference equal to 0.95. Let A denote the event that an inference for the population about men is correct. Let B represent the event of a corresponding inference about women being correct. Suppose that these are independent events.

- (a) Find the probability that (i) *both* inferences are correct, (ii) *neither* inference is correct.
- (b) Construct the probability distribution for Y = number of correct inferences.
- (c) With what probability would each inference need to be correct in order for the probability to be 0.95 that *both* are correct?

- (a) Find the probability that (i) *both* inferences are correct, (ii) *neither* inference is correct.

Using the multiplicative law, $P(AB) = P(A).P(B)$ since A and B are independent events. Total probability is 1. <https://stackoverflow.com/questions/15155778/superscript-in-markdown-github-flavored> => helped with superscripts in markdown files

Using $P(A^C) = 1 - P(A)$ rule,

$P(A)$ - men, $P(B)$ - women $P(A) = P(B) = 0.95$ So, $P(A^C) = P(B^C) = 0.05$

so, Both being right = $P(A).P(B) = 0.9025$ Both being wrong = $P(A^C).P(B^C) = 0.0025$

```
In [17]: import numpy as np

# Probability that an inference is correct
p = 0.95

# (i) Probability both inferences are correct
both_correct = p * p

# (ii) Probability neither inference is correct
# complement probability = 1 - p
neither_correct = (1 - p) * (1 - p)

print("(a)")
print("Probability both are correct =", both_correct)
print("Probability neither are correct =", neither_correct)
```

(a)

Probability both are correct = 0.9025

Probability neither are correct = 0.0025000000000000044

(b) Construct the probability distribution for Y = number of correct inferences.

There are 3 possibilities : $Y = 0$ (both wrong) $Y = 1$ (one of them is correct) $Y = 2$ (both are correct)

Note: using + to denote Union If A and B are disjoint, $P(A + B) = P(A) + P(B)$

Also, the events are independent, so $P(AB) = P(A).P(B)$

$P(B) = P(A) = 0.95$ Their compliments are 0.05.

$Y = 0$ (both wrong) is easy : Multiply both compliments

$Y=1$ basically, we need to multiply the the probabilities and compliments and then add both results.

Which would be $(p * (1 - p)) + ((1 - p) * p)$

$Y=2$ is easy. multiply both probabilities $P(A).P(B) = p*p$

Answer Probability Distribution of Y

$P(Y = 0) = 0.0025000000000000044$ $P(Y = 1) = 0.095000000000000008$ $P(Y = 2) = 0.9025$

```
In [18]: # (b) Construct probability distribution for Y = number of correct inferences

# Probability of both wrong
PY0 = (1 - p) * (1 - p)

# Probability of one correct
PY1 = (p * (1 - p)) + ((1 - p) * p)
```

```
# Probability of both correct
PY2 = p * p

print("\n(b)")
print("Probability Distribution of Y")
print("-----")
print("P(Y = 0) =", PY0)
print("P(Y = 1) =", PY1)
print("P(Y = 2) =", PY2)
```

(b)

Probability Distribution of Y

```
-----
P(Y = 0) = 0.002500000000000044
P(Y = 1) = 0.09500000000000008
P(Y = 2) = 0.9025
```

(c) With what probability would each inference need to be correct in order for the probability to be 0.95 that *both* are correct?

$P(A).P(B) = 0.95$.. Assuming both are equal, $p = P(A) = P(B)$

Their product is $p^2 = 0.95$

The sqrt is the answer : 0.9746794344808963

In [19]: # (c) Find probability each inference must have so that BOTH are correct wi

```
required_p = np.sqrt(0.95)
print("\n(c)")
print("Required probability =", required_p)
```

(c)

Required probability = 0.9746794344808963

Problem # 2.4.

A wine connoisseur is asked to match five glasses of red wine with the bottles from which they came, representing five different grape types.

(a) Set up a sample space for the five guesses.

(b) With random guessing, find the probability of getting all five correct.

(a) Set up a sample space for the five guesses.

Since this is permutation, and we have 5 grape types, $5! = 120$. So sample space will have 120 permutations (not combinations because change of position counts)

(b) With random guessing, find the probability of getting all five correct.

Probability of getting all five correct: 0.008333333333333333

```
In [20]: import math

# Total possible matchings
sample_space = math.factorial(5)

# Only one arrangement gets all 5 correct
favorable_outcomes = 1

probability = favorable_outcomes / sample_space

print("Probability of getting all five correct:")
print(probability)
```

Probability of getting all five correct:
0.008333333333333333

Problem # 2.15.

Each week an insurance company records Y = number of payments because of a home burning down. State conditions under which we would expect Y to approximately have a Poisson distribution.

****2.4.6 States, for Poisson , Counts of home fires in disjoint weeks are independent. It is essentially impossible to have two or more fires occurring simultaneously in an infinitesimally small time interval. The rate of home fires is approximately constant over time.**

mapping to our case would be No. of home fires this week does not affect number of fires next week In an extremely tiny instant of time, it is extremely unlikely multiple independent homes burn simultaneously Average number of home fires per week stays approximately constant over time**

Problem # 2.16.

Each day a hospital records the number of people who come to the emergency room for treatment.

(a) In the first week, the observations from Sunday to Saturday are 10, 8, 14, 7, 21, 44, 60. Do you think that the Poisson distribution might describe the random variability of this phenomenon adequately. Why or why not?

(b) Would you expect the Poisson distribution to better describe, or more poorly describe, the number of weekly admissions to the hospital for a rare disease? Why?

(a) In the first week, the observations from Sunday to Saturday are 10, 8, 14, 7, 21, 44, 60. Do you think that the Poisson distribution might describe the random variability of

this phenomenon adequately. Why or why not?

No, a single Poisson distribution with one constant rate probably does not adequately describe these daily ER counts. For the ER data, the variance is much larger than the mean, indicating considerably larger variability than the Poisson distribution allows. Therefore, the Poisson distribution the right one for this case. From the book, it sounds like Overdispersion

```
In [21]: import numpy as np

# Daily ER admissions
data = np.array([10, 8, 14, 7, 21, 44, 60])

# Mean
mean = np.mean(data)

# Variance
variance = np.var(data)

print("Mean =", mean)
print("Variance =", variance)
```

Mean = 23.428571428571427

Variance = 363.3877551020408

(b) Would you expect the Poisson distribution to better describe, or more poorly describe, the number of weekly admissions to the hospital for a rare disease? Why?

Yes, Poisson would likely be a better model for weekly admissions of a rare disease, because rare, independent events occurring at a roughly constant rate are exactly the type of phenomenon Poisson is designed for.

In []:

Problem # 2.17.

An instructor gives a course grade of B to students who have total score on exams and homeworks between 800 and 900, where the maximum possible is 1000. If the total scores have approximately a normal distribution with mean 830 and standard deviation 50, about what proportion of the students receive a B?

In paper, we can use the formula given in Section 2.3.6

$Z = Y - \text{mean} / \text{SD}$ (same as the one in yhe lab exercise where we called it z score)

when $Y = 800$, $Z = (800 - 830) / 50 = -0.6$ When 900 , it will be 1.4

so $-0.6 \leq Z \leq 1.4$

Then we can apply $\text{cdf}(1.4) - \text{cdf}(-0.6)$ using the standard normal CDF using python

which prints : Probability = 0.6449902230161553

I fed my solution to ChatGPT and asked for an alternate and it provided an easier solution. Good learning.

```
In [25]: from scipy.stats import norm
from scipy.stats import norm

# Given values
mean = 830
sd = 50

# Standardize the endpoints into Z-scores
z1 = (800 - mean) / sd # -0.6
z2 = (900 - mean) / sd # 1.4

# cdf(z2) - cdf(z1) using the standard normal CDF
probability = norm.cdf(z2) - norm.cdf(z1)

print(f"z1 = {z1}, z2 = {z2}")
print("Probability =", probability)

#### code from AI - simpler
# Given values
mu = 830
sigma = 50

# Probability between 800 and 900
probability = norm.cdf(900, mu, sigma) - norm.cdf(800, mu, sigma)

print("Probability =", probability)
```

```
z1 = -0.6, z2 = 1.4
Probability = 0.6449902230161553
Probability = 0.6449902230161553
```

Problem # 2.20.

Create a data file with the income values in the **Income** data file linked on the Module 2 Readings page.

(a) Construct a histogram or a smooth-curve approximation for the *pdf* of income in the corresponding population by plotting results using the density function in R (explained in Exercise 1.18).

(b) Of the probability distributions studied in this chapter, which do you think might be most appropriate for these data? Why? Plot the probability function of that distribution having the same mean and standard deviation as the income values. Does it seem to describe the income distribution well?

(a) Your answer goes here

```
In [32]: # (a) Construct a histogram or a smooth-curve approximation for the *pdf* of income data
import pandas as pd
import matplotlib.pyplot as plt

# Load the income data
df = pd.read_csv("Income.dat", sep='\\s+')

# Extract income column
income = df["income"]

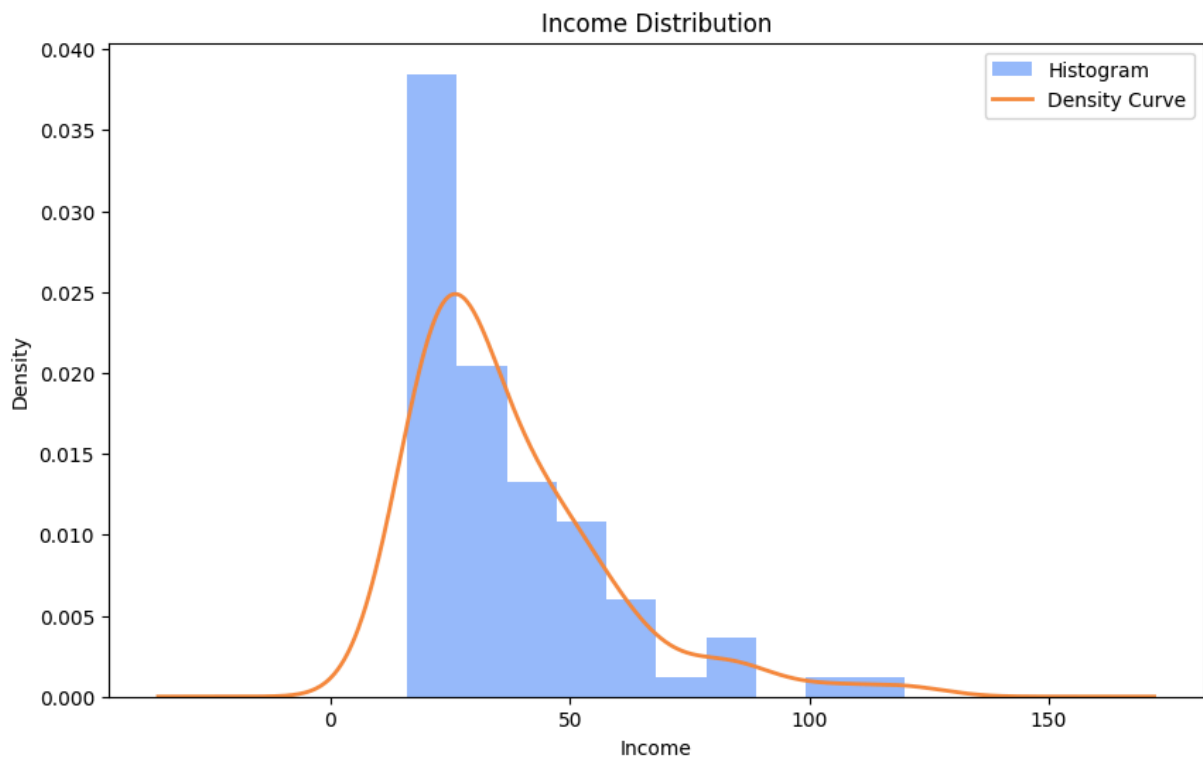
# Create histogram and smooth density curve
plt.figure(figsize=(10, 6))

# Histogram
plt.hist(income, bins=10, density=True, alpha=0.6, label="Histogram")

# (KDE)
income.plot(kind="kde", linewidth=2, label="Density Curve")

plt.xlabel("Income")
plt.ylabel("Density")
plt.title("Income Distribution")

plt.legend()
plt.show()
```



b) Of the probability distributions studied in this chapter, which do you think might be most appropriate for these data? Why? Plot the probability function of that distribution

having the same mean and standard deviation as the income values. Does it seem to describe the income distribution well?

The normal distribution does not describe the income data very well because the histogram is not symmetric and has a long right tail. Probably Gamma because it is right skewed and has a long tail, income can't be negative, etc.,

```
In [34]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm, lognorm

# Load the income data
df = pd.read_csv("Income.dat", sep='\\s+')
# Extract income column
income = df["income"]

# Compute mean and standard deviation
mean = income.mean()
sd = income.std()

print("Mean =", mean)
print("Standard Deviation =", sd)

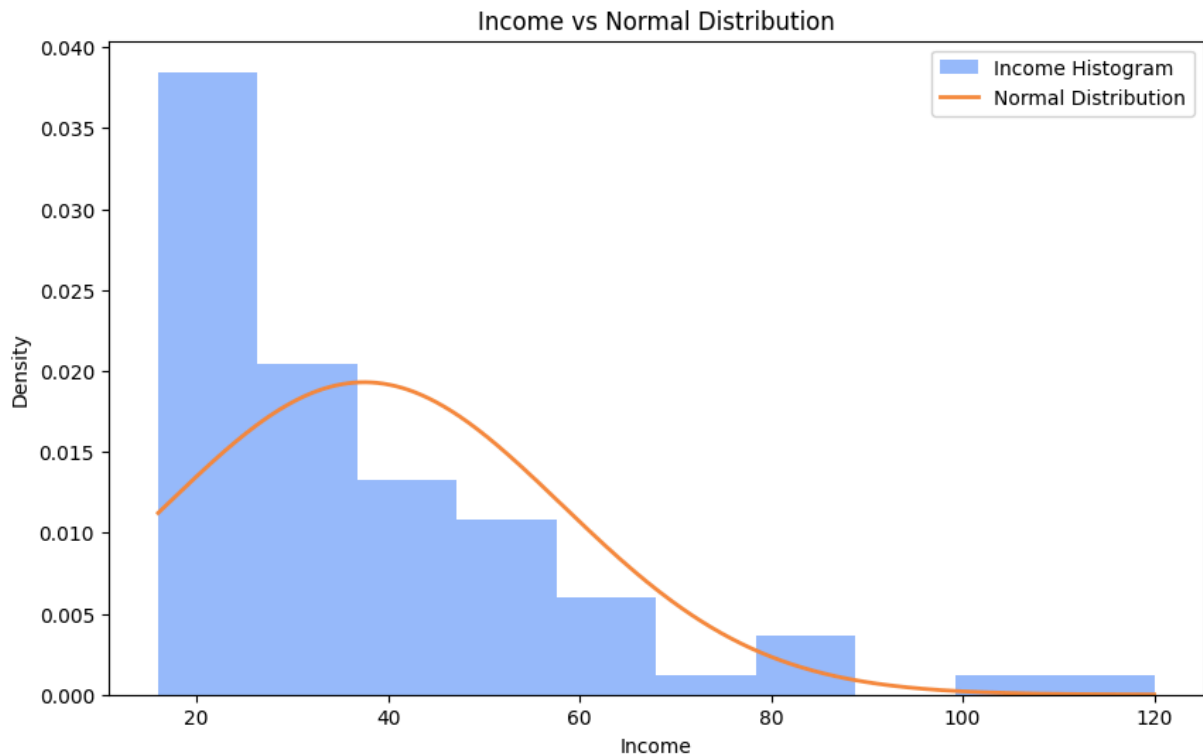
# Generate x values
x = np.linspace(income.min(), income.max(), 500)

# Normal Distribution
normal_pdf = norm.pdf(x, mean, sd)
plt.figure(figsize=(10, 6))
# Histogram of income data
plt.hist(income, bins=10, density=True, alpha=0.6, label="Income Histogram")

# Normal distribution curve
plt.plot(x, normal_pdf, linewidth=2, label="Normal Distribution")
plt.xlabel("Income")
plt.ylabel("Density")
plt.title("Income vs Normal Distribution")
plt.legend()
plt.show()
```

Mean = 37.525

Standard Deviation = 20.67284343519784



Problem # 2.21.

Plot the gamma distribution by fixing the shape parameter $k = 3$ and setting the scale parameter = 0.5, 1, 2, 3, 4, 5. What is the effect of increasing the scale parameter? (See also Exercise 2.48.)

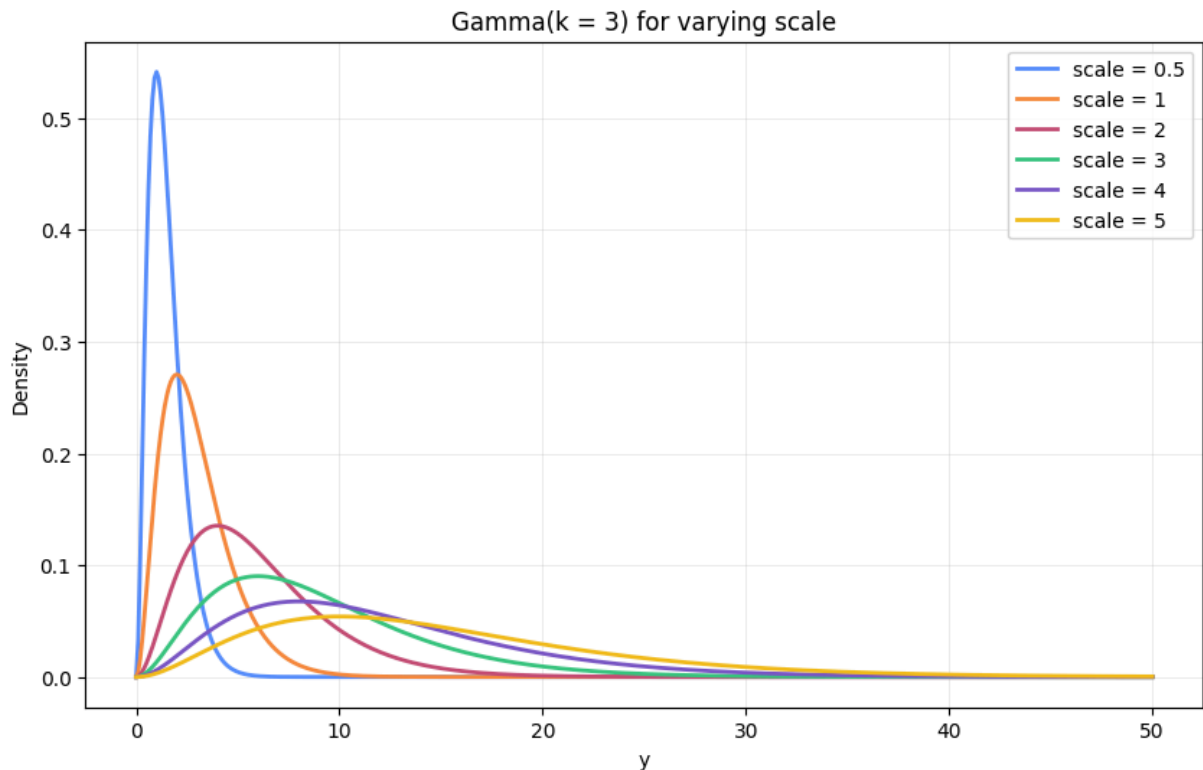
It appears that the shape does not change with scale.. Like zoomed in/out on x-axis

```
In [35]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gamma

k = 3 # fixed shape parameter
scales = [0.5, 1, 2, 3, 4, 5] # scale parameter values to compare
x = np.linspace(0, 50, 500)

plt.figure(figsize=(10, 6))
for s in scales:
    plt.plot(x, gamma.pdf(x, a=k, scale=s),
             linewidth=2, label=f"scale = {s}")

plt.xlabel("y")
plt.ylabel("Density")
plt.title("Gamma(k = 3) for varying scale")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



Problem # 2.22.

Consider the mammogram diagnostic example in Section 2.1.4.

(a) Show that the joint probability distribution of diagnosis and disease status is as shown in Table 2.6. Given that a diagnostic test result is positive, explain how this joint distribution shows that the 12% of incorrect diagnoses for the 99% of women not having breast cancer swamp the 86% of correct diagnoses for the 1% of women actually having breast cancer.

(b) The first test for detecting HIV-positive status had a sensitivity of 0.999 and specificity of 0.9999. Explain what these mean. If at that time 1 in 10,000 men were truly HIV-positive, find the positive predictive value. Based on this example, explain the potential disadvantage of routine diagnostic screening of a population for a rare disease.

Table 2.6. Joint probability distribution for disease status and diagnosis of breast-cancer mammogram, based on conditional probabilities in Table 2.1.

Disease Status	Positive (+)	Negative (−)	Total
Yes (D)	0.0086	0.0014	0.01
No (D^c)	0.1188	0.8712	0.99

(a) Show that the joint probability distribution of diagnosis and disease status is as shown in Table 2.6. Given that a diagnostic test result is positive, explain how this

joint distribution shows that the 12% of incorrect diagnoses for the 99% of women not having breast cancer swamp the 86% of correct diagnoses for the 1% of women actually having breast cancer.

Using the definition of conditional probability, $P(+ \text{ and } D) = P(+ | D) \cdot P(D)$, and similarly for the other three combinations:

$$P(+ \text{ and } D) = 0.86 \cdot 0.01 = 0.0086 \quad P(- \text{ and } D) = 0.14 \cdot 0.01 = 0.0014 \quad P(+ \text{ and } D^c) = 0.12 \cdot 0.99 = 0.1188 \quad P(- \text{ and } D^c) = 0.88 \cdot 0.99 = 0.8712$$

Given a positive test, the positive predictive value is $P(D | +) = P(+ \text{ and } D) / P(+) = 0.0086 / (0.0086 + 0.1188) = 0.0086 / 0.1274 = 0.0675\dots$

So only about 6.75% of positive results truly have cancer. The 12% false-positive rate acts on the 99% of women without cancer, producing 0.1188, which is far more than the 0.0086 from the 86% sensitivity acting on the 1% who actually have cancer.

(b) The first test for detecting HIV-positive status had a sensitivity of 0.999 and specificity of 0.9999. Explain what these mean. If at that time 1 in 10,000 men were truly HIV-positive, find the positive predictive value. Based on this example, explain the potential disadvantage of routine diagnostic screening of a population for a rare disease. Sensitivity 0.999 = 99.9% of HIV-positive men test positive. Specificity 0.9999 = 99.99% of HIV-negative men test negative.

With $P(\text{HIV}) = 1/10000$: $P(+) = (0.999)(0.0001) + (0.0001)(0.9999) = 0.0002$ $P(\text{HIV} | +) = (0.999 \times 0.0001) / 0.0002 = 0.50$ So only about 50% of positive results truly correspond to HIV. Disadvantage: for a rare disease, even a near-perfect test produces roughly as many false positives as true positives, so most positives are false alarms that cause unnecessary anxiety, follow-up testing, and cost when applied to the general population.

```
In [36]: # ----- Part (a): Mammogram -----

prevalence_of_cancer = 0.01 # 1% of women have breast cancer
prob_no_cancer = 1 - prevalence_of_cancer # 99%

sensitivity = 0.86 # P(positive | has cancer)
false_positive_rate = 0.12 # P(positive | no cancer)
specificity = 1 - false_positive_rate # 0.88

# Joint probabilities = conditional × marginal (Table 2.6)
prob_cancer_and_positive_test = sensitivity * prevalence_of_cancer # 0.0086
prob_cancer_and_negative_test = (1 - sensitivity) * prevalence_of_cancer # 0.0014
prob_no_cancer_and_positive_test = false_positive_rate * prob_no_cancer # 0.1188
prob_no_cancer_and_negative_test = specificity * prob_no_cancer # 0.8712

print("Joint distribution (Table 2.6):")
print(f"P(cancer, positive) = {prob_cancer_and_positive_test:.4f}")
print(f"P(cancer, negative) = {prob_cancer_and_negative_test:.4f}")
print(f"P(no cancer, positive) = {prob_no_cancer_and_positive_test:.4f}")
```

```

print(f"P(no cancer, negative) = {prob_no_cancer_and_negative_test:.4f}")

# Marginal probability of a positive test
prob_positive_test = ( prob_cancer_and_positive_test + prob_no_cancer_and_pos

# Bayes' Rule: P(cancer | positive test)
prob_cancer_given_positive_test = (
    prob_cancer_and_positive_test / prob_positive_test
)

print(f"\nP(positive test)           = {prob_positive_test:.4f}")
print(f"P(cancer | positive test)     = {prob_cancer_given_positive_test:.4f}
      f"(~{prob_cancer_given_positive_test * 100:.2f}%)")

# ----- Part (b): HIV screening -----

prevalence_of_hiv = 1 / 10_000 # 1 in 10,000 men are HIV-positive
prob_not_hiv = 1 - prevalence_of_hiv

hiv_test_sensitivity = 0.999 # P(positive | HIV-positive)
hiv_test_specificity = 0.9999 # P(negative | HIV-negative)
hiv_test_false_positive_rate = 1 - hiv_test_specificity

prob_hiv_test_positive = (
    hiv_test_sensitivity * prevalence_of_hiv
    + hiv_test_false_positive_rate * prob_not_hiv
)

positive_predictive_value = (
    (hiv_test_sensitivity * prevalence_of_hiv) / prob_hiv_test_positive
)

print("\nHIV screening:")
print(f"  P(test positive)           = {prob_hiv_test_positive:.6f}")
print(f"  PPV = P(HIV | test positive) = {positive_predictive_value:.4f}
      f"(~{positive_predictive_value * 100:.2f}%)")

```

Joint distribution (Table 2.6):

$P(\text{cancer, positive}) = 0.0086$
 $P(\text{cancer, negative}) = 0.0014$
 $P(\text{no cancer, positive}) = 0.1188$
 $P(\text{no cancer, negative}) = 0.8712$

$P(\text{positive test}) = 0.1274$
 $P(\text{cancer} \mid \text{positive test}) = 0.0675$ (~6.75%)

HIV screening:
 $P(\text{test positive}) = 0.000200$
 $\text{PPV} = P(\text{HIV} \mid \text{test positive}) = 0.4998$ (~49.98%)

Problem # 2.27.

The distribution of X = heights (cm) of women in the U.K. is approximately $N(162, 7^2)$.
 Conditional on $X = x$, suppose Y = weight (kg) has a $N(3.0 + 0.40x, 8^2)$ distribution.

Simulate and plot 1000 observations from this approximate bivariate normal distribution. Approximate the marginal means and standard deviations for X and Y . Approximate and interpret the correlation.

Your answer goes here

Bayes' Theorem and AI Decision Reliability

An autonomous drone uses an onboard AI to detect whether an area contains a **target object** (for example, a missing person wearing a bright jacket).

The system raises an alert when it believes a target is present. However, the model is not perfect.

- $P(\text{Target}) = 0.03$
- $P(\text{Alert} \mid \text{Target}) = 0.95$
- $P(\text{Alert} \mid \text{No Target}) = 0.10$

Tasks

1. Compute the probability of an alert

Use the law of total probability:

$$P(\text{Alert}) = P(\text{Alert} \mid \text{Target})P(\text{Target}) + P(\text{Alert} \mid \text{No Target})P(\text{No Target})$$

2. Compute the posterior probability

Apply Bayes' theorem.

$$P(\text{Target} \mid \text{Alert}) = \frac{P(\text{Alert} \mid \text{Target})P(\text{Target})}{P(\text{Alert})}$$

3. Implement in Python

Write a short script that calculates and prints both probabilities with clear labels.

4. Interpret your result

In a short paragraph (as a code comment), explain what your computed $(P(\text{Target} \mid \text{Alert}))$ means about the AI's reliability when it triggers an alert.

5. Bonus

Create a function that allows a user to change:

- $P(\text{Target})$
- $P(\text{Alert} \mid \text{Target})$
- $P(\text{Alert} \mid \text{No Target})$

Then return the updated posterior probability and show how the result changes when $P(\text{Target}) = 0.005$.

Starter Code

```
In [37]: ## The code seems to be already working, so, I tried to interpret the result
# P(Alert): 0.1255
# P(Target | Alert): 0.2271
# Posterior with P(Target)=0.005: 0.04556354916067146

# P(Alert): 0.1255 & P(Target | Alert): 0.2271 means that when the drone raises an alert, there is a 22.71% chance that a target is actually present.

## Bayes' Theorem in AI Detection Reliability

##### Given probabilities #####
P_target = 0.03
P_alert_given_target = 0.95
P_alert_given_no_target = 0.10
#####
# Compute complements
P_no_target = 1 - P_target

# 1. Total probability of alert
P_alert = (P_alert_given_target * P_target) + (P_alert_given_no_target * P_no_target)

# 2. Posterior probability using Bayes' theorem
P_target_given_alert = (P_alert_given_target * P_target) / P_alert

print(f"P(Alert): {P_alert:.4f}")
print(f"P(Target | Alert): {P_target_given_alert:.4f}")

# Interpretation:
# When the AI raises an alert, this tells us how likely it is that
# a target is actually present. A low posterior means many false alarms.

# Bonus: parameterized function
def bayes_posterior(P_target, P_alert_given_target, P_alert_given_no_target):
    P_no_target = 1 - P_target
    P_alert = (P_alert_given_target * P_target) + (P_alert_given_no_target * P_no_target)
    return (P_alert_given_target * P_target) / P_alert

print("Posterior with P(Target)=0.005:",
      bayes_posterior(0.005, 0.95, 0.10))
```

P(Alert): 0.1255

P(Target | Alert): 0.2271

Posterior with P(Target)=0.005: 0.04556354916067146